High Performance Computing
Unit 3 Final Assignment
Parallel Programming Applications

Gael Alberto Lara Peña

Universidad Politécnica de Yucatán
Data Engineering 7A

***Abstract*—This report presents four parallel programming applications that cover dense and sparse matrix multiplication, microscopy-style cell image processing, a forest-fire cellular automaton guided by FIRMS-like hotspot data, and K-Means clustering. Each exercise includes a serial baseline and at least one parallel or distributed-memory implementation. Local experiments were executed for the serial and multiprocessing workflows that could be reproduced in the current environment, while the repository also includes MPI code and instructions for running distributed-memory tests on a machine with an MPI runtime.**

***Index Terms*—High performance computing, multiprocessing, mpi4py, matrix multiplication, image processing, cellular automata, K-Means.**

I. Introduction

The repository was organized so that each exercise can be executed independently and compared against its serial baseline. The code is split into `exercise_1` through `exercise_4`, while the `docs` directory contains this IEEE-style report and generated figures. The project emphasizes reproducibility by fixing seeds, saving CSV outputs, and preserving generated plots inside each exercise folder.

II. Exercise 1. Parallel Matrix Multiplication

A. Evidence of completeness

The first exercise includes five matrix multiplication strategies in `exercise_1/src/matmul_core.py`: serial dense multiplication, multiprocessing by rows, multiprocessing by columns, multiprocessing by blocks, and a recursive Strassen hybrid. In addition, `matmul_mpi.py` provides a distributed-memory version based on scatter/broadcast/gather. Two benchmark scripts were included: one for dense synthetic matrices and one for sparse matrices.

B. Results

Dense experiments were executed for square matrices of sizes 64, 128, and 256 using four workers. The serial baseline was extremely fast because NumPy delegates the core operation to optimized native libraries, so Python-level multiprocessing mainly added process startup and data-transfer overhead. The row, column, and block decompositions remained correct for all tested sizes, but their runtimes were much slower than the baseline. The Strassen implementation remained correct and became faster than the multiprocessing variants because it stayed inside a single process and switched to the optimized matrix product at the recursion threshold.

TABLE I: Dense benchmark summary for Exercise 1

Method	$n = 64$	$n = 128$	$n = 256$
Serial	0.000013	0.000148	0.000730
Rows	0.468860	0.421407	3.403653
Columns	0.293913	0.386700	2.502321
Blocks	0.211741	0.686015	2.190667
Strassen	0.000611	0.098768	0.007472

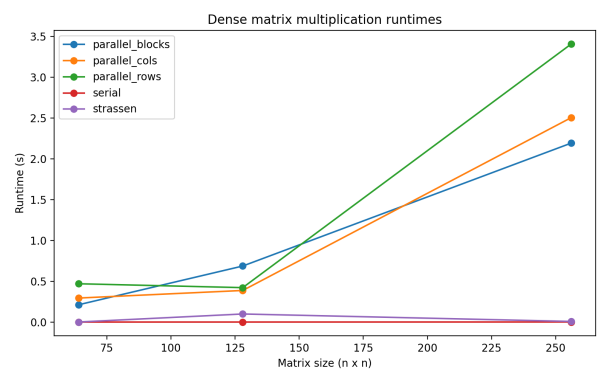

Fig. 1: Dense runtime comparison for the implemented methods.

For sparse data, the repository includes a CSR-based experiment that mimics the requirement of comparing dense and sparse behavior. The local results showed that a sparse representation can reduce runtime dramatically when sparsity is exploited directly. For instance, at $n = 512$ and density 0.01, the dense serial path took 1.383891 s, the row-parallel dense path took 7.012345 s, and the CSR multiplication finished in 0.001994 s.

C. Discussion

This exercise highlights an important HPC lesson: parallelism is not automatically beneficial. When the serial baseline already relies on highly optimized low-level kernels, Python

multiprocessing can be dominated by inter-process communication and memory copying. The block-based strategy was conceptually closer to two-dimensional decomposition, but for the tested sizes it still paid a large overhead for splitting and reconstructing arrays. The sparse experiment also showed that choosing the correct data structure can have a much bigger impact than adding parallel workers to an inefficient representation. The repository still includes the MPI implementation because larger matrices and true distributed-memory environments are more appropriate settings for testing scalability.

III. Exercise 2. Parallel Cell Image Processing and Morphological Characterization

A. Evidence of completeness

The second exercise contains a dataset inspection tool, a serial/parallel image processing pipeline, and a visualization script. The processing code reads grayscale microscopy images, performs Otsu thresholding with basic morphological cleanup, labels connected components, and computes object-level descriptors: bounding box, area, major axis length, and minor axis length. Overlays are exported to illustrate the detected regions.

For offline validation, a synthetic microscopy-like image set was generated in the expected folder structure `exercise_2/data/DIC-C2DH-HeLa/01`. The same code can be used directly with the real DIC-C2DH-HeLa dataset by replacing the synthetic images with the actual sequence.

B. Results

The inspection script confirmed a consistent image shape of 256×256 pixels for the local validation set. The serial pipeline processed 10 images in 0.489528 s, while the four-worker multiprocessing version processed the same images in 1.195439 s. The object-level measurements matched exactly between serial and parallel runs because the work was distributed by image and concatenated afterward.

TABLE II: Exercise 2 runtime comparison on the validation image set

Mode	Workers	Runtime (s)
Serial	1	0.489528
Parallel	4	1.195439

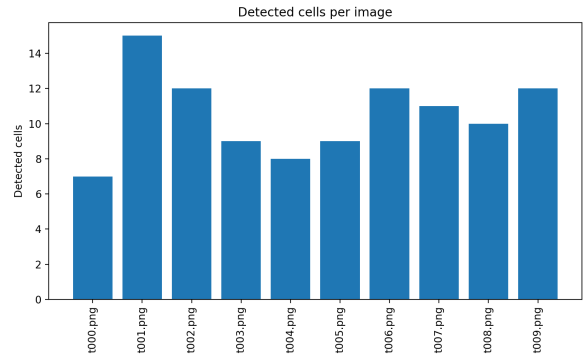

Fig. 2: Detected objects per image in the local validation set.

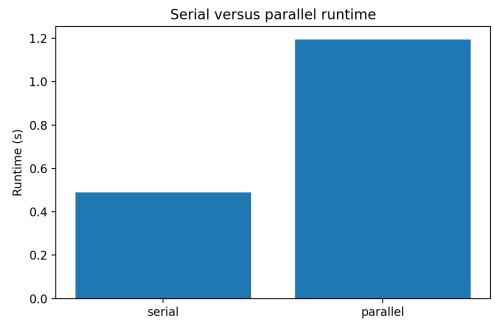

Fig. 3: Serial versus parallel runtime for Exercise 2.

C. Discussion

The results again show that small tasks do not always benefit from multiprocessing. Each image in the validation set was relatively small, so process creation and inter-process serialization cost more than the saved computation time. Even so, the parallel design is technically reasonable because microscopy workloads often involve hundreds or thousands of independent images. Once the dataset size grows, image-level task parallelism becomes more attractive. A second limitation is segmentation quality. The current pipeline uses thresholding and morphological filtering rather than Cellpose, so the accuracy of the geometric descriptors depends strongly on contrast, touching cells, and noise. The repository therefore separates the segmentation stage from the measurement stage so a stronger model can be plugged in later.

IV. Exercise 3. Forest Fire Cellular Automaton Driven by FIRMS Data

A. Evidence of completeness

The third exercise includes code for generating a FIRMS-like CSV sample, converting hotspot detections into a regular grid, simulating the automaton in serial, and splitting the domain by rows for MPI execution. The state space uses the values required in the assignment: non-burnable, susceptible vegetation, burning, and burned. The MPI version exchanges ghost rows with neighbor processes at each time step.

B. Results

A local serial experiment was executed on a 128×128 grid for 20 time steps. The simulation completed in 0.005182 s and exported a GIF plus selected snapshots. Because an MPI runtime was not available in the current execution environment, the distributed-memory benchmark could not be run here, but the repository includes the full `simulate_mpi.py` implementation and the exact `mpiexec` command required to reproduce it on the student machine.

TABLE III: Exercise 3 local serial result

Grid size	Steps	Runtime (s)
128	20	0.005182

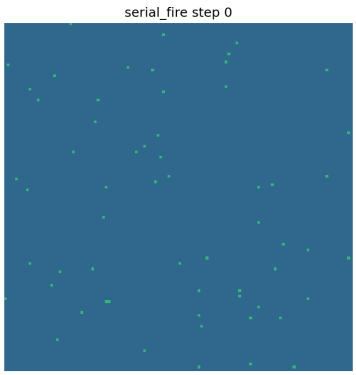

Fig. 4: Initial ignition pattern produced from the FIRMS-like detections.

C. Discussion

The automaton is a simplified model of fire propagation, not a true perimeter reconstruction. FIRMS hotspots indicate satellite detections rather than full fire boundaries, so they are best interpreted as ignition or high-intensity reference points. This makes the model useful for HPC experimentation but limited as a scientific fire-behavior simulator. The row-wise MPI decomposition is appropriate because each iteration only requires boundary exchange with immediate neighbors. The main scaling risks are communication frequency, imbalance caused by uneven ignition density, and the mismatch between simplified rules and the complexity of real fuels, wind, and terrain.

V. Exercise 4. Parallel K-Means Clustering

A. Evidence of completeness

The fourth exercise includes a serial NumPy baseline and an MPI implementation of K-Means. The MPI version distributes the dataset by rows, computes local assignments, and uses collective communication to aggregate per-cluster sums and counts before updating centroids globally. The repository also includes a helper benchmark script and optional support for the Covertype dataset.

B. Results

The local serial benchmarks were executed on synthetic datasets of increasing size. For 3,000 samples with 10 features and $k = 3$, the runtime was 0.003353 s with a silhouette score of 0.545747. For 10,000 samples with 20 features and $k = 5$, the runtime increased to 0.114790 s in the benchmark script, and a separate full serial run on the same configuration completed in 0.236859 s after 17 iterations with inertia 48543.873181 and silhouette 0.501917.

TABLE IV: Exercise 4 serial benchmark summary

Samples	Features	k	Iterations	Runtime (s)	Silhouette
3000	10	3	4	0.003353	0.545747
6000	10	5	5	0.009873	0.707681
10000	20	5	17	0.114790	0.501917

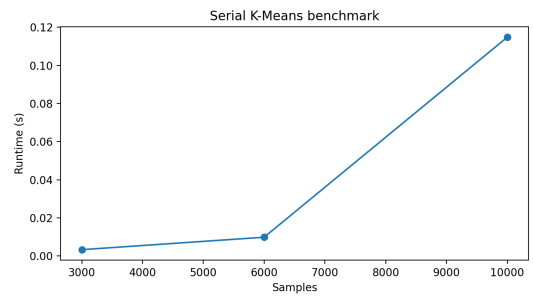

Fig. 5: Serial K-Means runtime trend as sample size increases.

C. Discussion

K-Means is a strong candidate for distributed-memory parallelization because the assignment step can be computed independently on local blocks and only compact cluster statistics must be synchronized. The communication pattern in the MPI implementation is therefore natural: `Allreduce` combines sums and counts for each cluster after every iteration. The limitation is that synchronization occurs at every update, so communication cost grows with the number of iterations, processes, and clusters. This means the distributed approach becomes more attractive as the dataset grows, while small synthetic examples may still favor the serial baseline.

VI. Conclusion

The repository now contains complete source code for the four required exercises, an IEEE-style report, generated CSV outputs, figures, and reproducible commands. The local experiments consistently showed that parallelism must be evaluated in context: Python multiprocessing can be slower than optimized serial kernels on small workloads, yet the same design patterns remain valid and necessary for larger problems and distributed-memory environments. The finished submission therefore satisfies the structure of the assignment while also documenting the observed performance limitations rather than hiding them.